Screenshot 1a

```
┌──(pes1ug24cs213㉿Rock)-[~/PES1UG24CS213-pes-vcs]
└─$ make test_objects && ./test_objects
make: 'test_objects' is up to date.
Stored blob with hash: d58213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
Object stored at: .pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
PASS: blob storage
PASS: deduplication
PASS: integrity check

All Phase 1 tests passed.
```

Screenshot 1b

```
┌──(pes1ug24cs213㉿Rock)-[~/PES1UG24CS213-pes-vcs]
└─$ find .pes/objects -type f | sort | head -n 20
.pes/objects/25/ef1fa07ea68a52f800dc80756ee6b7ae34b337afedb9b46a1af8e11ec4f476
.pes/objects/2a/594d39232787fba8eb7287418aec99c8fc2ecdaf5aaf2e650eda471e566fcf
.pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
```

Screenshot 2a

```
┌──(pes1ug24cs213㉿Rock)-[~/PES1UG24CS213-pes-vcs]
└─$ make test_tree && ./test_tree
gcc -o test_tree test_tree.o object.o tree.o index.o -lcrypto
Serialized tree: 139 bytes
PASS: tree serialize/parse roundtrip
PASS: tree deterministic serialization

All Phase 2 tests passed.
```

Screenshot 2b

```
┌──(pes1ug24cs213㉿Rock)-[~/PES1UG24CS213-pes-vcs]
└─$ xxd .pes/objects/67/77b79ffd64aef8c1da0f2df5025afbee060ccd3c6a6599257d6f644e683547 | head -n 20
00000000: 7472 6565 2039 3200 3130 3036 3434 2066  tree 92.100644 f
00000010: 312e 7478 7400 f3fc 011d 7161 a922 e757  1.txt.....qa.".W
00000020: 0913 ca1e 46bb 5aa3 d4c1 0c1a 7ca7 dc32  ....F.Z.....|..2
00000030: afbd 9a1c bf45 3130 3036 3434 2066 322e  .....E100644 f2.
00000040: 7478 7400 9f91 9199 749e 58a2 2748 d939  txt.....t.X.'H.9
00000050: 0495 d7c1 62ee 6e71 cc48 e6ca cebe 471b  ....b.nq.H....G.
00000060: da65 5d58                                .e]X
```

Screenshot 3a

```
┌──(pes1ug24cs213㉿Rock)-[~/PES1UG24CS213-pes-vcs]
└─$ ./pes init
echo "test file" > f1.txt
./pes add f1.txt
./pes status
Initialized empty PES repository in .pes/
Staged changes:
  staged:     f1.txt

Unstaged changes:
  (nothing to show)

Untracked files:
  untracked:  .gitignore
  untracked:  object.c
  untracked:  commit.c
  untracked:  commit.h
  untracked:  test_objects.c
  untracked:  test_objects
  untracked:  README.md
  untracked:  test_tree.c
  untracked:  tree.c
  untracked:  f2.txt
  untracked:  Makefile
  untracked:  pes.h
  untracked:  GEMINI.md
  untracked:  test_sequence.sh
  untracked:  index.c
  untracked:  index.h
  untracked:  tree.h
  untracked:  test_tree
  untracked:  pes.c
```

Screenshot 3b

```
┌──(pes1ug24cs213㉿Rock)-[~/PES1UG24CS213-pes-vcs]
└─$ cat .pes/index
100644 325f340305db25f49f06f49a9915b080ef94ea67ccca8560a4a07d52037eb22b 1776752637 10 f1.txt
```

Screenshot 4a

```
┌──(pes1ug24cs213㉿Rock)-[~/PES1UG24CS213-pes-vcs]
└─$ ./pes log
commit 62c916a8004858236c206d98e5b3194e2094666d3043d93a7d2ef265212248e2
Author: PES User <pes@localhost>
Date:   1776752761

    Add second file and update first⌖G

commit e1267e91ba8e7dbdc2a16f2f2bf8538ccf1da3dd1218f5463132c27b17309046
Author: PES User <pes@localhost>
Date:   1776752760

    Initial commit
```

Screenshot 4b

```
┌──(pes1ug24cs213㉿Rock)-[~/PES1UG24CS213-pes-vcs]
└─$ find .pes -type f | sort
.pes/HEAD
.pes/index
.pes/objects/04/ba0ca3fe68a43f06180dc83a5f9667fde291c4de935043e73051ecfe31d07c
.pes/objects/32/5f340305db25f49f06f49a9915b080ef94ea67ccca8560a4a07d52037eb22b
.pes/objects/62/c916a8004858236c206d98e5b3194e2094666d3043d93a7d2ef265212248e2
.pes/objects/67/77b79ffd64aef8c1da0f2df5025afbee060ccd3c6a6599257d6f644e683547
.pes/objects/9f/919199749e58a22748d9390495d7c162ee6e71cc48e6cacebe471bda655d58
.pes/objects/e1/267e91ba8e7dbdc2a16f2f2bf8538ccf1da3dd1218f5463132c27b17309046
.pes/objects/f3/fc011d7161a922e7570913ca1e46bb5aa3d4c10c1a7ca7dc32afbd9a1cbf45
.pes/objects/f6/b92871b97059b445cb99a8276ba6d68141dc6e399af91407832d64e5dbee20
.pes/refs/heads/main
```

Screenshot 4c

```
┌──(pes1ug24cs213㉿Rock)-[~/PES1UG24CS213-pes-vcs]
└─$ cat .pes/HEAD
cat .pes/refs/heads/main
ref: refs/heads/main
62c916a8004858236c206d98e5b3194e2094666d3043d93a7d2ef265212248e2
```

Final

```
┌──(pes1ug24cs213㉿Rock)-[~/PES1UG24CS213-pes-vcs]
└─$ make test-integration
=== Running integration tests ===
bash test_sequence.sh
=== PES-VCS Integration Test ===

--- Repository Initialization ---
Initialized empty PES repository in .pes/
PASS: .pes/objects exists
PASS: .pes/refs/heads exists
PASS: .pes/HEAD exists

--- Staging Files ---
Status after add:
Staged changes:
  staged:     file.txt
  staged:     hello.txt

Unstaged changes:
  (nothing to show)

Untracked files:
  (nothing to show)


--- First Commit ---
Committed: d51355de9720... Initial commit

Log after first commit:
commit d51355de9720ef0a71dc33db2ca24a3df749e15470c31da36741ed6cdf537bfc
Author: PES User <pes@localhost>
Date:   1776753335

    Initial commit


--- Second Commit ---
Committed: c986dd27b7bc... Update file.txt

--- Third Commit ---
Committed: b6cc4176f52e... Add farewell

--- Full History ---
commit b6cc4176f52e3b10ec616624797fadff64ab6dcd3507ad6e0c67ad904ee3c9b5
Author: PES User <pes@localhost>
Date:   1776753335

    Add farewell

commit c986dd27b7bc3bb96c2c4648dedaf9ae6fd8d8f6da7664a3523b08fb008aecf3
Author: PES User <pes@localhost>
Date:   1776753335

    Update file.txt

commit d51355de9720ef0a71dc33db2ca24a3df749e15470c31da36741ed6cdf537bfc
Author: PES User <pes@localhost>
Date:   1776753335

    Initial commit


--- Reference Chain ---
HEAD:
ref: refs/heads/main
refs/heads/main:
b6cc4176f52e3b10ec616624797fadff64ab6dcd3507ad6e0c67ad904ee3c9b5

--- Object Store ---
Objects created:
10
.pes/objects/0b/d69098bd9b9cc5934a610ab65da429b525361147faa7b5b922919e9a23143d
.pes/objects/10/1f28a12274233d119fd3c8d7c7216054ddb5605f3bae21c6fb6ee3c4c7cbfa
.pes/objects/58/a67ed1c161a4e89a110968310fe31e39920ef68d4c7c7e0d6695797533f50d
.pes/objects/ab/5824a9ec1ef505b5480b3e37cd50d9c80be55012cabe0ca572dbf959788299
.pes/objects/b1/7b838c5951aa88c09635c5895ef7e08f7fa1974d901ce282f30e08de0ccd92
.pes/objects/b6/cc4176f52e3b10ec616624797fadff64ab6dcd3507ad6e0c67ad904ee3c9b5
.pes/objects/c9/86dd27b7bc3bb96c2c4648dedaf9ae6fd8d8f6da7664a3523b08fb008aecf3
.pes/objects/d0/7733c25d2d137b7574be8c5542b562bf48bafeaa3829f61f75b8d10d5350f9
.pes/objects/d5/1355de9720ef0a71dc33db2ca24a3df749e15470c31da36741ed6cdf537bfc
.pes/objects/db/07a1451ca9544dbf66d769b505377d765efae7adc6b97b75cc9d2b3b3da6ff

=== All integration tests completed ===
```

Phase 5: Branching and Checkout

Q5.1: How would you implement pes checkout <branch>? What files change, and what happens to the working directory?

Solution: To implement checkout, the following steps must occur:

1. **Update HEAD:** The file .pes/HEAD must be updated to point to the new branch (e.g., ref: refs/heads/<branch_name>).
2. **Read the Tree:** Locate the latest commit hash from the branch file, find that commit object, and retrieve its root tree hash.
3. **Working Directory Sync:** The most complex part. You must delete all currently tracked files in the working directory and replace them with the blobs defined in the target commit's tree.
4. **Complexity:** It is dangerous because of **"dirty" changes**. If a user has modified a file but hasn't committed it, a checkout could overwrite their work. You must perform a "safety check" to ensure no unsaved work is lost before switching.

Q5.2: How would you detect a "dirty working directory" conflict using only the index and the object store?

Solution: You compare three states:

1. **The Index:** What the VCS *thinks* the file looks like.
2. **The Working Directory:** The actual file on disk.
3. **The Object Store (HEAD):** What was in the last commit.

If the mtime or size of a file in the working directory differs from its entry in the **Index**, the file is "dirty." If that same file is also different in the target branch you are switching to, a conflict exists, and the VCS should refuse the checkout.

Q5.3: What happens if you make commits in a "Detached HEAD" state? How could a user recover those commits?

Solution: In a Detached HEAD state, .pes/HEAD contains a raw commit hash instead of a reference to a branch.

- **Commits:** You can still commit. New commit objects are created, and HEAD updates to the new hash. However, no branch (like main) is updated.
- **The Risk:** If you checkout back to main, you lose the pointer to those new commits. They become "orphaned."
- **Recovery:** The user would need to find the hash of the last "detached" commit (using git reflog in real Git, or by searching the .pes/objects folder in ours) and manually create a new branch pointing to that hash.

Phase 6: Garbage Collection (GC)

Q6.1: Describe an algorithm to find and delete unreachable objects. What data structure would you use?

Solution: The algorithm is **Mark-and-Sweep**:

1. **Mark Phase:** Start at all "Roots" (all files in .pes/refs/heads/ and the hash in .pes/HEAD).
2. **Traverse:** Recursively follow all pointers: Commits point to Trees and Parents; Trees point to Blobs and Sub-trees.
3. **Data Structure:** Use a **Hash Set** (or a Bloom Filter for very large repos) to store the IDs of all "reachable" objects.
4. **Sweep Phase:** Iterate through every file in .pes/objects/. If its hash is NOT in your "reachable" set, delete it.
5. **Estimate:** For 50 branches and 100k commits, you would visit at least 100,000 commit objects, plus their associated trees. Total objects visited could easily exceed 500k.

Q6.2: Why is it dangerous to run GC concurrently with a commit? Describe a race condition.

Solution: GC is dangerous during a commit because of the **"Unreferenced Object" race condition**.

- **Scenario:** A pes commit is in progress. It has just written a new Blob to the object store but hasn't yet written the Commit object that points to it.
- **The Race:** If the GC runs at this exact microsecond, it looks at all existing branches, doesn't see a pointer to that new Blob yet, and deletes it as "garbage."
- **The Failure:** When the commit process finally tries to finish and reference that Blob, the object is gone, causing repository corruption.
- **Git's Solution:** Real Git uses a **grace period** (usually 2 weeks). GC will never delete an object that was created very recently, even if it appears unreachable, to ensure concurrent operations can finish safely.